

Протокол HTTP

Вызовы удалённых серверов

Григорий Вахмистров



Проверка связи

Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти

Поставьте в чат:



если меня видно и слышно



если нет

Рекомендации



Если смотрите с компьютера

- Используйте браузеры Google Chrome или Microsoft Edge
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**



Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение на телефоне

Правила участия

- 1 Приготовьте блокнот и ручку, чтобы записывать важные мысли и идеи
- 2 Продолжительность вебинара — 2 часа
- 3 Вы можете писать свои вопросы в чате
- 4 Запись вебинара будет доступна в вашем личном кабинете
- 5 Оффтоп обсуждения можно продолжить в учебном чате

Григорий Вахмистров

О спикере:

- в разработке с 2015 года
- весь опыт связан с MSA
- предпочитаю Event-Driven Architecture



Цели занятия

- Разберёмся, из чего состоят HTTP-запрос и ответ: методы, заголовки, тело, коды статусов
- Узнаем, зачем в Java используют современные HTTP-клиенты вместо сокетов, и обсудим их преимущества
- Познакомимся с Apache HttpClient 5.x и выясним, чем он лучше версии 4.5
- Рассмотрим пример отправки GET-запроса через HttpClient 5.x
- Увидим, как преобразовывать JSON-ответ в Java-объекты с помощью Jackson и зачем нужны `@JsonProperty` и `ObjectMapper`
- Обсудим, в чём разница между HTTP и HTTPS, и почему шифрование критично для безопасности

План занятия

- 1 Основы протоколов и HTTP
- 2 Современные HTTP-клиенты в Java
- 3 Apache HttpClient 5.x
- 4 Демонстрация GET-запроса
- 5 Работа с JSON и Jackson
- 6 HTTP и HTTPS
- 7 Итоги

Основы протоколов и HTTP

**Как вы понимаете значение слова
«протокол»?
Напишите свой вариант в чат**

Значения слова «протокол»



В обычной жизни

- запись заседания или собрания
- официальный акт или документ
- свод правил, например дипломатический



В IT

- набор правил, по которым компьютеры обмениваются данными в сети

Протоколы в IT

TCP и UDP — примеры транспортных протоколов

✓ Они обеспечивают:

- 1 доставку данных между компьютерами
- 2 указание размера передаваемых данных
- 3 описание отправителя и получателя

✓ Почему это важно

- 1 HTTP работает поверх TCP (а иногда UDP, например в HTTP/3)
- 2 без транспортных протоколов невозможно наладить обмен данными между клиентом и сервером

Протоколы в IT

Если нарушить порядок байт в TCP или UDP

- данные не дойдут до адресата

Если использовать только TCP или UDP

- разработчику придётся каждый раз придумывать свой формат передачи
- другим приложениям будет сложно подключаться и понимать эти данные

Почему этого недостаточно

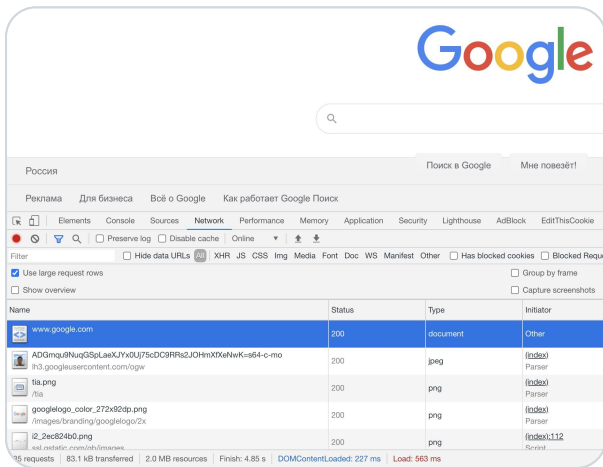
- такой подход неудобен для разработки корпоративных и веб-приложений
- поэтому появились протоколы прикладного уровня — они стандартизируют обмен данными

Популярные протоколы прикладного уровня

- HTTP — протокол передачи гипертекста, основа веба
- FTP — протокол передачи файлов
- DNS — протокол преобразования доменных имён в IP-адреса
- POP3 и SMTP — протоколы обмена электронной почтой
- SSH и TELNET — протоколы удалённого доступа к компьютеру или приложению

Протокол HTTP

- HTTP — базовый прикладной протокол для обмена данными в Интернете
- С его помощью браузер получает текст страниц, изображения, видео и другие ресурсы
- Каждый раз, когда вы открываете сайт, браузер отправляет HTTP-запросы к серверу
- В консоли браузера можно увидеть список всех HTTP-запросов и ответов от сервера
- ✓ Сегодня мы рассмотрим структуру HTTP-запроса и ответа подробнее



Источник: сайт google.com

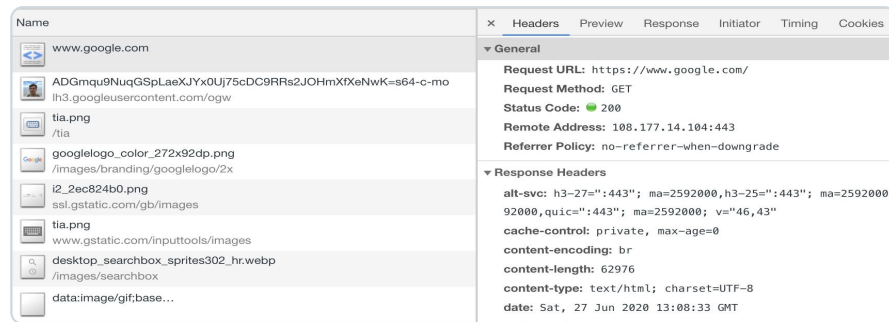
Скриншот отражает состояние сайта на момент подготовки материала, содержание и оформление могут измениться

Протокол HTTP

→ Если открыть любой запрос в панели разработчика браузера, можно увидеть его детали

→ В них отображаются

- URL и метод запроса
- код ответа от сервера
- заголовки и другая служебная информация

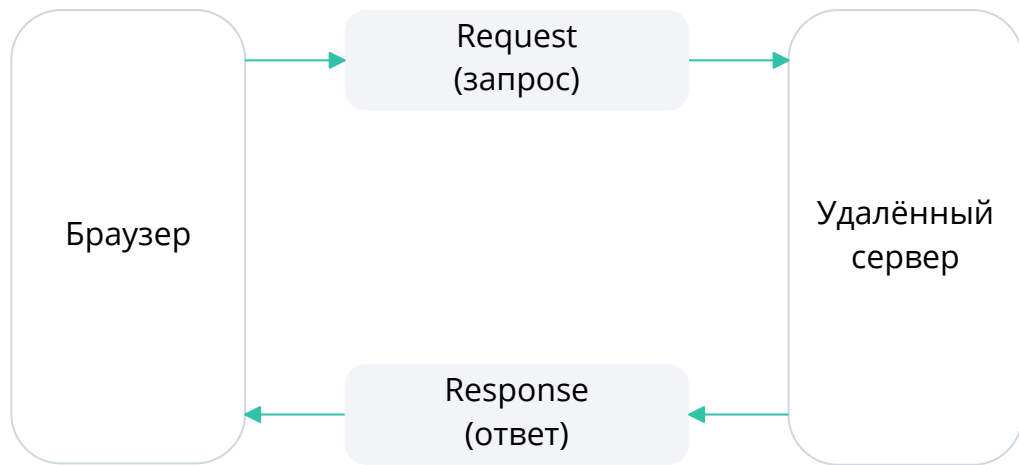


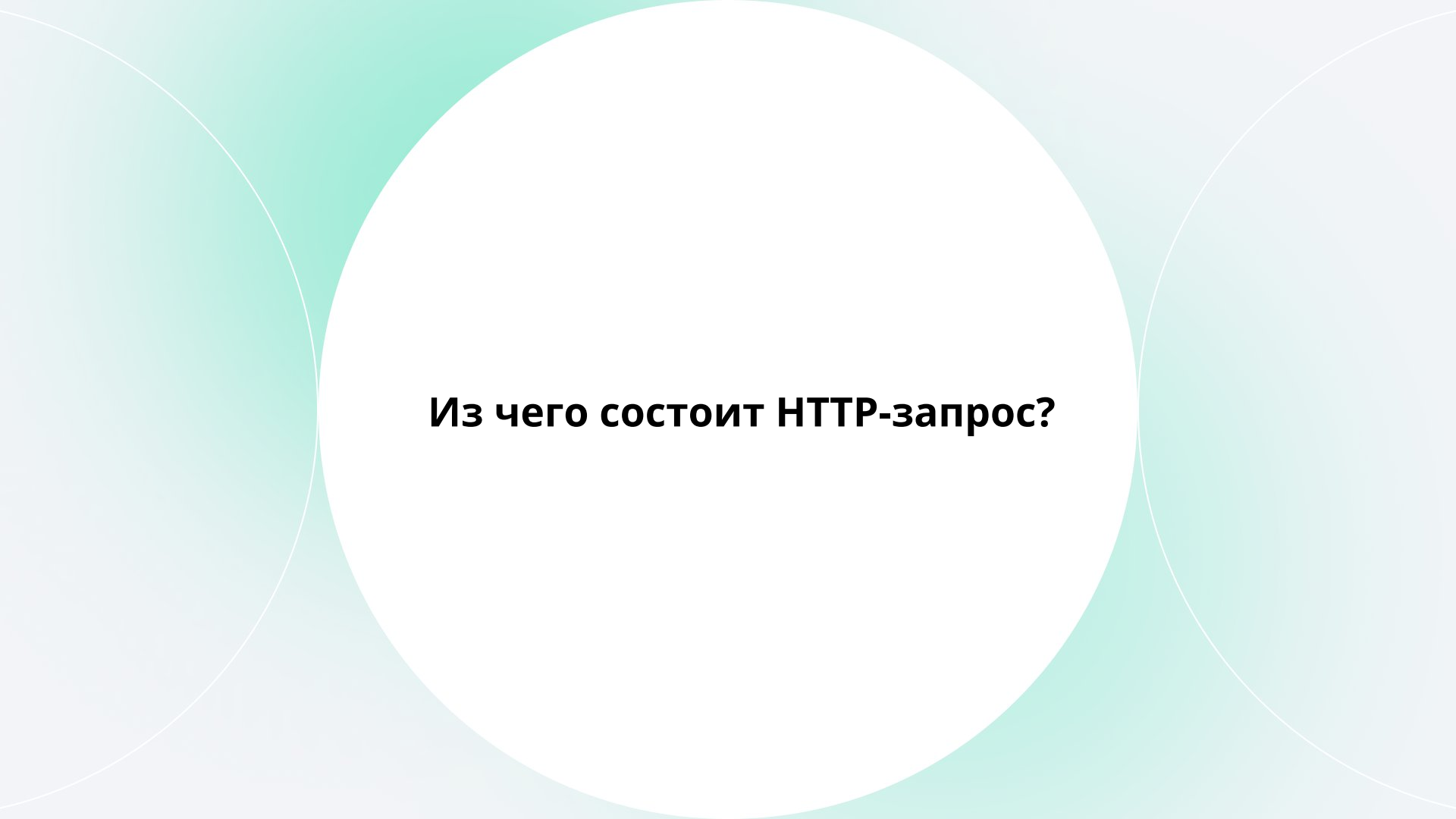
Источник: сайт google.com

Скриншот отражает состояние сайта на момент подготовки материала, содержание и оформление могут измениться

Описание протокола HTTP

- HTTP — клиент-серверный протокол
- Клиент (например, браузер) отправляет запрос к серверу
- Сервер обрабатывает запрос и возвращает ответ
- На месте браузера может быть любое приложение, которое умеет работать с HTTP





Из чего состоит HTTP-запрос?

HTTP-запрос (request)

Состоит из трёх частей

- 1 Стартовая строка (Request line) задаёт метод, адрес ресурса и версию протокола
- 2 Заголовки (Headers) дополнительные параметры передачи и служебная информация
- 3 Тело сообщения (Message Body) основные данные запроса, отделяется пустой строкой

Стартовая строка HTTP-запроса

Состоит из трёх элементов

- 1 Метод запроса (GET, POST и др.)
- 2 Путь к ресурсу (например, /index.html)
- 3 Версия протокола (HTTP/1.1, HTTP/2)

GET

/index.html

HTTP/1.1

Метод

Путь к ресурсу

Версия протокола

Собери запрос

- **Задание:** ваш браузер запрашивает главную страницу сайта yandex.ru по протоколу HTTP/1.1. Напишите в чат стартовую строку этого GET-запроса

Собери запрос

Правильный ответ:

```
GET / HTTP/1.1
```

Методы HTTP-запросов

- GET — получение данных с сервера
- POST — добавление новых данных
- PUT — полная замена данных
- PATCH — частичное обновление данных
- DELETE — удаление данных
- HEAD — как GET, но без тела ответа
- CONNECT — создание защищённого туннеля
- OPTIONS — запрос поддерживаемых методов
- TRACE — трассировка запроса и отладка

Путь к ресурсу

- Путь указывает, какой ресурс на сервере мы хотим получить
- Примеры ресурсов: веб-страница, файл, картинка, видео
- В путь можно добавить параметры запроса после знака ?
- Формат:

```
имя1=значение1&имя2=значение2
```

Пример пути с параметрами

```
/index.html?name=ivan&surname=semenov
```

Версия протокола

HTTP/1.1 — самая распространённая версия сегодня

- 1 HTTP/2 — современный стандарт (с 2015 года), активно используется наряду с 1.1
- 2 HTTP/3 — новая версия, основанная на протоколе QUIC

HTTP-запрос: заголовки

- Заголовки содержат дополнительные параметры запроса (язык, тип данных, авторизация и др.)
- Начиная с HTTP/1.1, обязательным стал заголовок Host — в нём указывается имя сервера, к которому отправляется запрос

→ Пример

```
GET /index.html?name=ivan&surname=semenov HTTP/1.1  
Host: google.com
```

HTTP-запрос: заголовки

На практике чаще всего используются заголовки:

- 1 Host — адрес сервера, к которому обращается клиент
- 2 User-Agent — информация о клиенте (браузер, ОС)
- 3 Accept — какие типы данных клиент готов принять
- 4 Content-Type — формат тела запроса (например, application/json)
- 5 Authorization — данные для аутентификации пользователя

HTTP-запрос: часто используемые заголовки

- 1 **User-Agent** — информация о клиенте (браузер, ОС)

Пример: `User-Agent: Mozilla/5.0 (...)`

- 2 **Authorization** — данные для авторизации

Пример: `Authorization: Basic AKJjns31jkl==`

- 3 **Accept** — форматы данных, которые клиент готов принять

Пример: `Accept: text/html, application/xml`

- 4 **Accept-Language** — язык клиента

Пример: `Accept-Language: en-US`

- 5 **Cookie** — сохранённые данные сессии

Пример: `Cookie: PHPSESSID=r2t5uvjq435r4q7ib3vtdqj120`

Пример запроса с заголовками

Так выглядит реальный HTTP-запрос, отправленный браузером на сервер

```
GET /index.html HTTP/1.1
Host: google.com
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 6.1; en-US; rv:1.9.1.5)
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Connection: keep-alive
Cookie: PHPSESSID=r2t5uvjq435r4q7ib3vtdqj120
Pragma: no-cache
Cache-Control: no-cache
```

Пример запроса приведён в учебных целях. Реальные заголовки могут отличаться в зависимости от браузера и настроек сервера

Запрос. Тело

- 1 Тело запроса — необязательная часть HTTP-запроса, которая используется для передачи данных (текста, JSON, файлов и других форматов)
 - 2 Добавление данных в запрос обозначается заголовками
 - 3 Content-Length — длина тела запроса в байтах
 - 4 Content-Type — тип передаваемого содержимого, например application/json, text/html, multipart/form-data
 - 5 Content-Encoding — указывает, сжато ли тело запроса (gzip, deflate и др.)
- Когда вы загружаете файлы на Google Drive, Dropbox или другое хранилище, именно тело запроса передаёт эти данные

Запрос. Тело

Request Line

```
GET /index.html HTTP/1.1
```

Заголовки

```
Content-Type: application/json; charset=utf-8
```

Тело запроса

```
{"id": "25", "product": "143"}
```

Свой HTTP-метод

- Создать собственный HTTP-метод технически возможно, но это плохая практика
- Для работы с новым методом придётся писать собственный клиент, так как стандартные клиенты его не поддерживают
- Такой подход не рекомендуется

HTTP-ответ (response)

Формат ответа похож на формат запроса и состоит из трёх частей

- 1 Стартовая строка (Status line) — указывает версию протокола, код состояния и пояснение к нему
- 2 Заголовки (Headers) — дополнительная служебная информация об ответе
- 3 Тело сообщения (Message Body) — сами данные ответа (страница, JSON, файл и т. д.)

Ответ. Стартовая строка

Стартовая строка ответа состоит из трёх элементов

- 1 Версия протокола — например, HTTP/1.1
- 2 Код ответа — например, 200
- 3 Пояснение — текстовое описание кода, например OK



Пример:

```
HTTP/1.1 200 OK
```

Ответ. Стартовая строка



Коды ответа делятся на группы

- 100–199 информационные
- 200–299 успешные
- 300–399 перенаправления
- 400–499 клиентские ошибки
- 500–599 серверные ошибки

**С какими популярными кодами
ответов вы сталкивались?**

Ответ. Популярные коды ответа

- 200 OK — запрос выполнен успешно
- 301 Moved Permanently — ресурс перемещён на постоянной основе
- 302 Found — ресурс временно перемещён
- 400 Bad Request — неверный синтаксис запроса
- 401 Unauthorized — требуется аутентификация
- 404 Not Found — ресурс не найден
- 405 Method Not Allowed — метод не поддерживается сервером
- 500 Internal Server Error — внутренняя ошибка сервера
- 503 Service Unavailable — сервис недоступен

Ответ. Заголовки

→ Как и в запросах, заголовки в ответах не обязательны, но несут дополнительную информацию

→ Используются для:

1 Описание типа ответа

`Content-Type: application/json, text/plain` и др.

2 Указание адреса для перенаправления (коды 301 и 302)

`Location: http://yandex.ru/index.html`

3 Хранение данных для последующих запросов

`Set-Cookie: session_id=wfsfasfasfadsdf; HttpOnly`

4 Существует множество других заголовков, используемых в разных сценариях.

Справочник: [MDN Web Docs — HTTP headers](#)

Ответ. Тело

→ **Тело ответа — необязательный элемент. Используется для передачи данных (текст, JSON или другой формат)**

→ **Основные заголовки:**

- `Content-Length` — длина тела ответа в байтах
- `Content-Type` — тип содержимого, например `application/json`, `text/html`, `multipart/form-data`
- `Content-Encoding` — способ сжатия тела (`gzip`, `deflate`, `br`)

Ответ. Тело

Status line

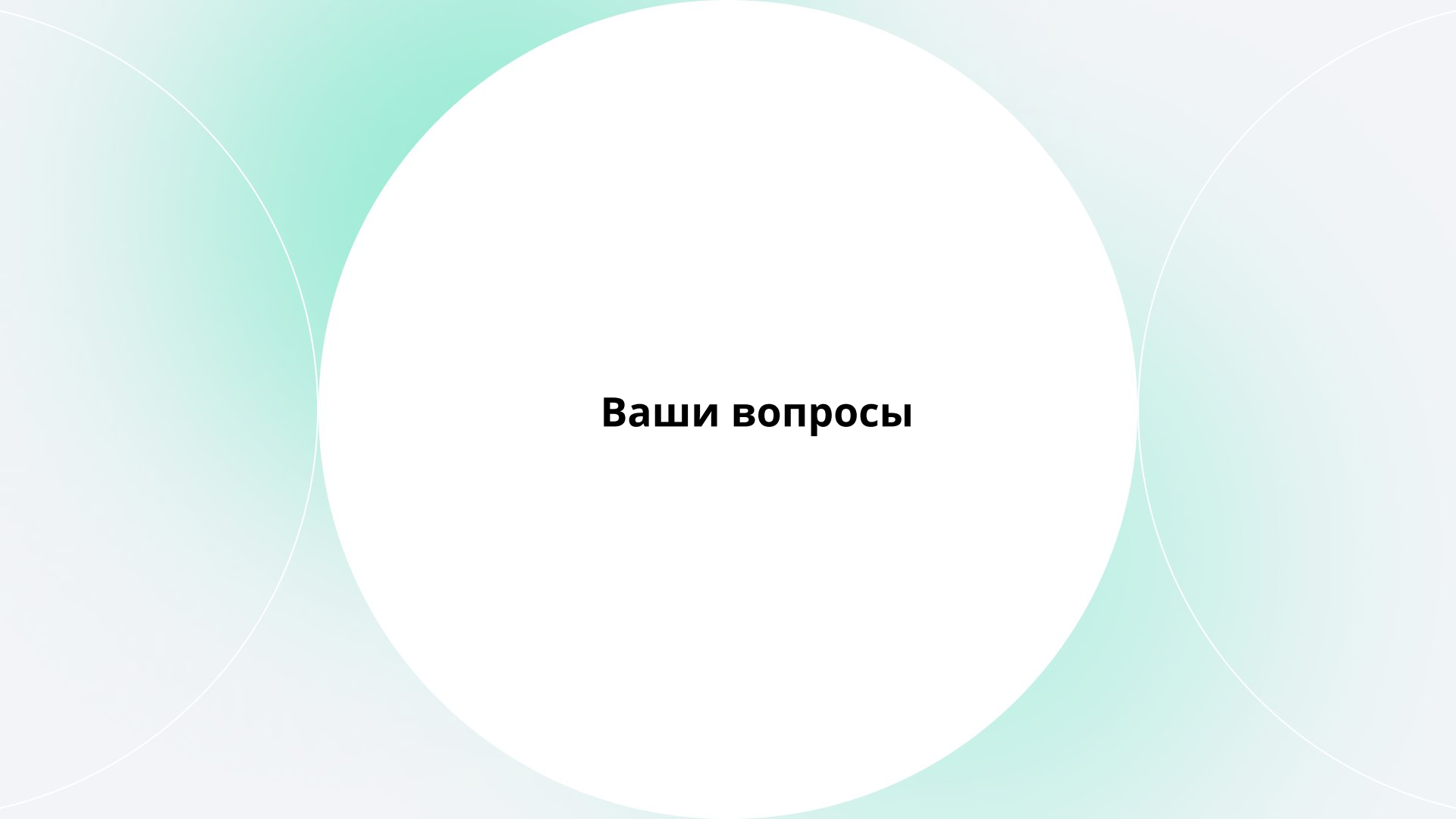
`HTTP/1.1 200 OK`

Заголовки

`Content-Type: application/json; charset=utf-8`

Тело ответа

`{"product": 143, "cost": 300}`



Ваши вопросы

Современные HTTP- клиенты в Java

Способы отправки HTTP-запроса в Java

Для работы с HTTP-запросами можно использовать разные библиотеки и встроенные классы:

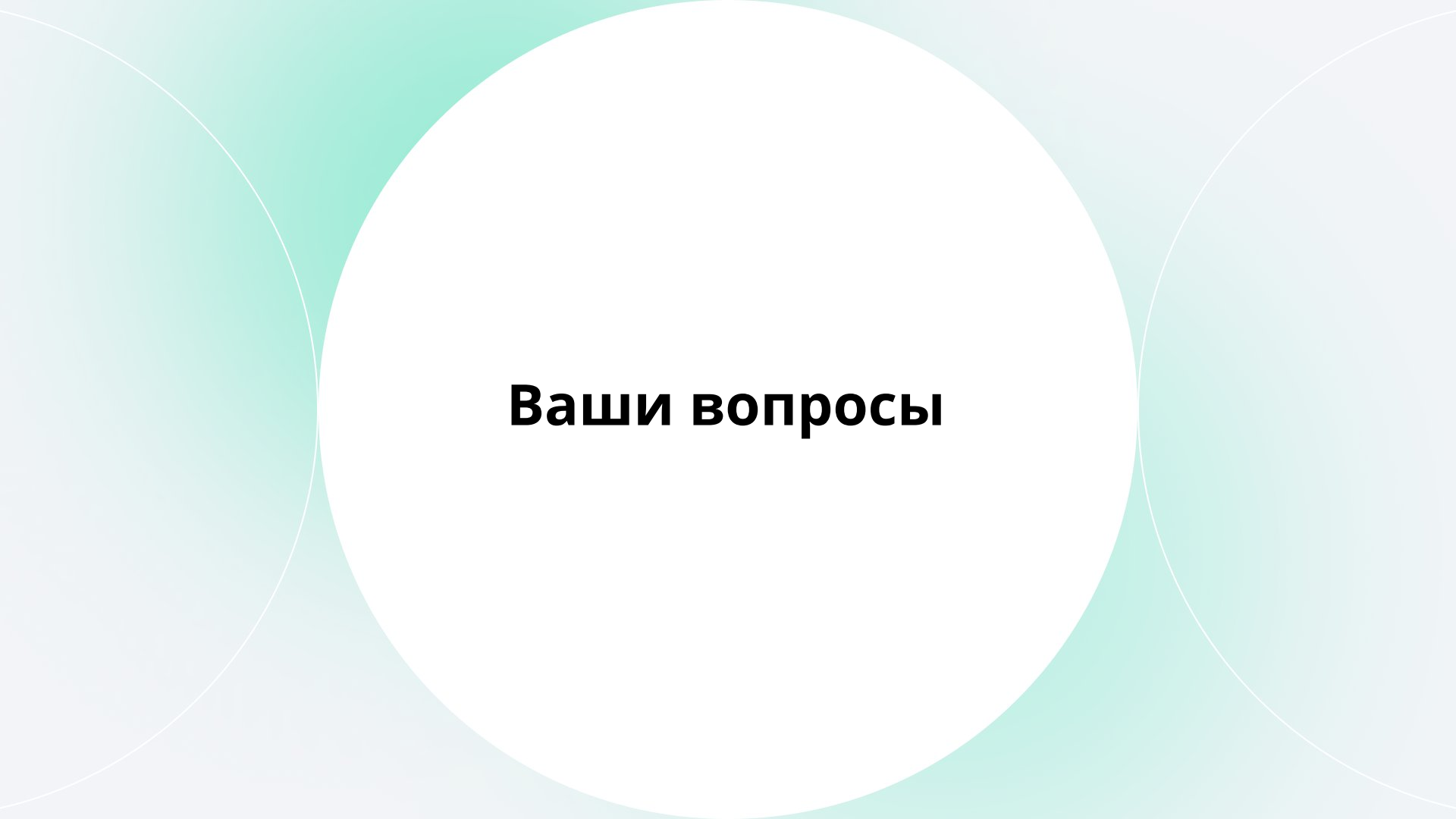
- 1 Apache HttpClient — одна из самых популярных библиотек, гибкая настройка
 - 2 OkHttp — современная библиотека, часто применяется в Android-разработке
 - 3 JDK HttpClient (с версии 11) — стандартное решение в Java, поддерживает HTTP/2
 - 4 JDK HttpURLConnection (с версии 1.1) — устаревший способ, не рекомендуется, так как нет поддержки HTTP/2
- Эти инструменты решают одну задачу, но отличаются по уровню удобства и набору возможностей. На практике чаще всего используют Apache HttpClient и OkHttp

Сравнение HttpClient 4.x и 5.x

Критерий	HttpClient 4.x	HttpClient 5.x
Пакет	<code>org.apache.http.*</code>	<code>org.apache.hc.client5.*</code> , <code>org.apache.hc.core5.*</code>
Архитектура	Монолитная (HTTP и клиентская логика смешаны)	Модульная (<code>core</code> , <code>client</code> , <code>async</code> отдельно)
API	Императивное, устаревшее, много шаблонного кода	Современное, <code>fluent-API</code> , <code>builder-подход</code>
Асинхронность	<code>CloseableHttpAsyncClient</code> (ограниченно)	Новый <code>HttpAsyncClients</code> с <code>Future</code> и <code>Callback</code>
TLS/SSL	На старом API Java SSL	Новая гибкая конфигурация (<code>TlsStrategy</code> , <code>PoolingAsyncClientConnectionManager</code>)

Сравнение HttpClient 4.x и 5.x

Критерий	HttpClient 4.x	HttpClient 5.x
HTTP/2	Нет поддержки	Поддержка из коробки
Proxy, Auth, Redirects	Реализованы, но не унифицированы	Унифицированная конфигурация через RequestConfig и HttpClientBuilder
Dependency	httpclient + httpcore	httpclient5, httpcore5 (чёткое разделение)
Логирование	Commons Logging / SLF4J (вручную)	SLF4J 2.x по умолчанию
Entity API	HttpEntity, EntityUtils	Новый API: ClassicHttpRequest, ClassicHttpResponse, StringEntity, InputStreamEntity
Connection pooling	PoolingHttpClientConnectionManager	Новый PoolingHttpClientConnectionManager Builder с расширенными стратегиями
Request execution API	HttpClient.execute(HttpUriRequest)	ClassicHttpRequests.create("GET", URI) + HttpClient.createDefault()



Ваши вопросы

Apache HttpClient 5.x

Создание и настройка Java-клиента

1 Создаём Maven-проект и добавляем зависимость:

```
<dependency>
  <groupId>org.apache.httpcomponents.client5</groupId>
  <artifactId>httpclient5</artifactId>
  <version>5.1.1</version>
</dependency>
```

2 HttpClient 5 использует логирование. Чтобы не возникало предупреждений при запуске проекта, добавляем зависимости для SLF4J и Log4j:

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>2.0.13</version>
</dependency>

<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.25.2</version>
</dependency>
```

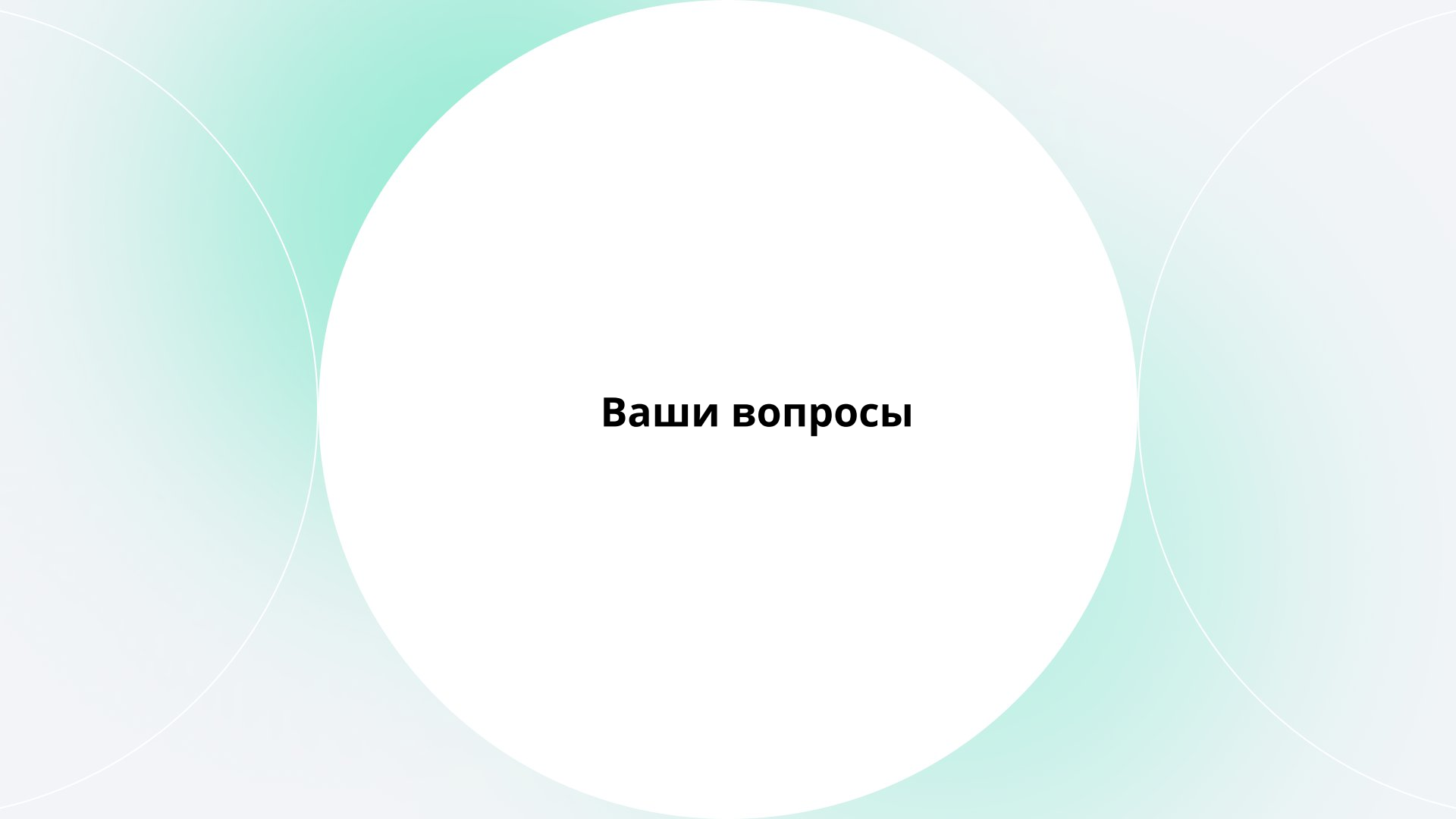
```
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-slf4j2-impl</artifactId>
  <version>2.25.2</version>
</dependency>
```

Создание и настройка Java-клиента

3 Создаём Main-класс и инициализируем HttpClient через HttpClientBuilder

```
import org.apache.http.client5.http.config.RequestConfig;
import org.apache.http.client5.http.impl.classic.CloseableHttpClient;
import org.apache.http.client5.http.impl.classic.HttpClientBuilder;
import java.io.IOException;
import java.util.concurrent.TimeUnit;

public class Main {
    public static void main(String[] args) throws IOException {
        CloseableHttpClient httpClient = HttpClientBuilder.create()
            .setUserAgent("My Test Service")
            .setDefaultRequestConfig(RequestConfig.custom()
                .setConnectTimeout(5, TimeUnit.SECONDS)
                .setResponseTimeout(30, TimeUnit.SECONDS)
                .setRedirectsEnabled(false)
                .build())
            .build();
    }
}
```

Ваши вопросы

Демонстрация GET-запроса

Вызов удалённого сервера

→ Вызовем сервер по адресу:

```
https://jsonplaceholder.typicode.com/posts
```

Чтобы сделать запрос, нужно:

1. Создать объект клиента HttpClient
2. Подготовить объект запроса HttpGet и добавить заголовки
3. Отправить запрос методом execute()
4. Получить и обработать ответ (заголовки, тело ответа)

Вызов удалённого сервера

```
public static final String REMOTE_SERVICE_URI = "https://jsonplaceholder.typicode.com/posts";

public static void main(String[] args) throws IOException {
    CloseableHttpClient httpClient = HttpClientBuilder.create().build();

    // создаём объект запроса
    HttpGet request = new HttpGet(REMOTE_SERVICE_URI);
    request.setHeader(HttpHeaders.ACCEPT, ContentType.APPLICATION_JSON.getMimeType());

    // отправляем запрос
    CloseableHttpResponse response = httpClient.execute(request);

    // выводим заголовки
    Arrays.stream(response.getHeaders())
        .forEach(System.out::println);

    // читаем тело ответа
    String body = new String(response.getEntity()
        .getContent()
        .readAllBytes(), StandardCharsets.UTF_8);
    System.out.println(body);
}
```

Вывод заголовков ответа от сервера

```
Date: Mon, 21 Oct 2024 07:28:00 GMT
Content-Type: application/json; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
Set-Cookie: sessionId=abc123; HttpOnly
X-Powered-By: Express
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 998
X-RateLimit-Reset: 1634816400
Vary: Accept-Encoding
Access-Control-Allow-Credentials: true
Cache-Control: max-age=43200
Pragma: no-cache
Expires: -1
X-Content-Type-Options: nosniff
ETag: W/"12345-67890"
Via: 1.1 varnish
```

- Date: Mon, 21 Oct 2024 07:28:00 GMT
→ время формирования ответа сервером
- Content-Type: application/json; charset=utf-8
→ тип возвращаемых данных (здесь JSON)
- Transfer-Encoding: chunked
→ способ передачи тела ответа (частями)
- Connection: keep-alive
→ соединение не закрывается сразу, можно отправлять новые запросы

Разбор заголовков ответа

- Set-Cookie: sessionId=abc123; HttpOnly
→ сервер устанавливает cookie у клиента
- X-Powered-By: Express
→ технология, на которой работает сервер
- X-RateLimit-Limit: 1000
→ лимит запросов за период
- X-RateLimit-Remaining: 998
→ сколько запросов ещё можно выполнить
- X-RateLimit-Reset: 1634816400
→ время, когда лимит обнулится
- Vary: Accept-Encoding
→ заголовки, влияющие на кэширование
- Access-Control-Allow-Credentials: true
→ разрешает пересылку cookies и авторизационных данных
- Cache-Control: max-age=43200
→ хранить ответ 12 часов
- Pragma: no-cache / Expires: -1
→ устаревшие директивы кэширования
- X-Content-Type-Options: nosniff
→ защита от подмены MIME-типа
- ETag: W/"12345-67890"
→ уникальный идентификатор версии ресурса
- Via: 1.1 varnish
→ цепочка прокси или шлюзов, через которые прошёл ответ

Вывод тела ответа от сервера

```
[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et\ncum\nreprehenderit molestiae"
  },
  {
    "userId": 1,
    "id": 2,
    "title": "qui est esse",
    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis"
  }
]
```

Пояснение

- Сервер возвращает список постов пользователей в формате JSON
- Это текстовое представление данных: ключи и значения сгруппированы в объекты

Почему важно преобразование: для работы в Java JSON-ответ нужно преобразовать:

- из строки в объекты
- сохранить данные в коллекцию (список, массив)
- использовать для дальнейшей логики программы

Практика

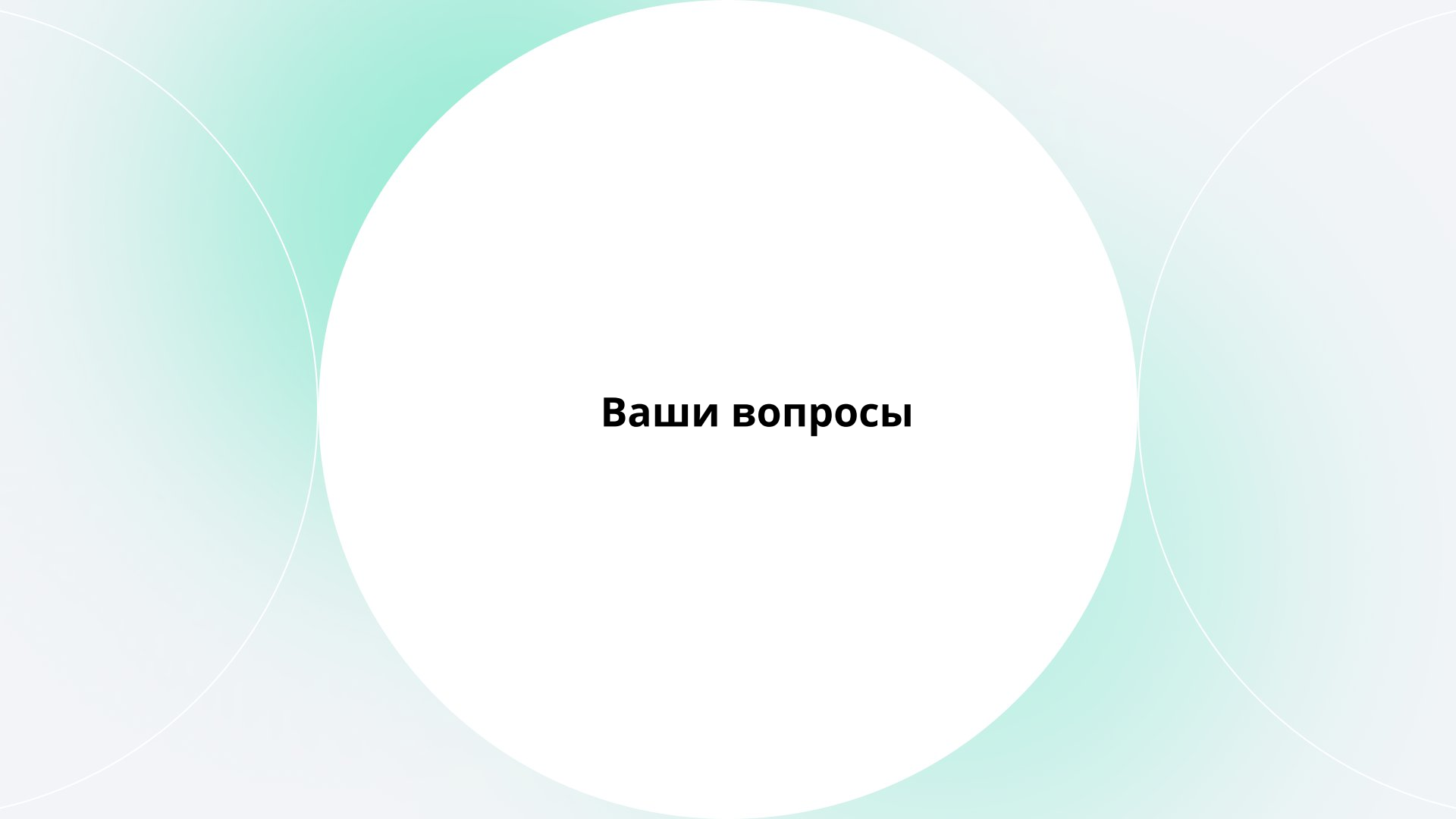
У вас есть заготовка Java-программы, которая отправляет GET-запрос к удалённому серверу. Заготовка по ссылке [здесь](#).

Цель: отправить GET-запрос и вывести статус + тело ответа

Ваша задача за 10 минут:

- Добавить заголовок **Accept: application/json**
- Вывести в консоль код статуса ответа
- Прочитать и вывести тело ответа как строку
- Не забудьте импорты, если они нужны

После выполнения — сравните ваш код с другими решениями



Ваши вопросы

Работа с JSON и Jackson

Преобразование JSON в Java-объекты

Чтобы преобразовать JSON в Java, используется библиотека Jackson (или Gson, но Jackson чаще применяют на backend)

→ **Добавляем зависимость в Maven:**

```
<dependency>

<groupId>com.fasterxml.jackson.core</group
Id>

    <artifactId>jackson-
databind</artifactId>
    <version>3.0.2</version>
</dependency>
```

→ **Класс Post.java для маппинга JSON:**

```
public class Post {
    private int userId;
    private int id;
    private String title;
    private String body;

    // геттеры и сеттеры
}
```

Класс Post.java

```
import com.fasterxml.jackson.annotation.JsonProperty;

public class Post {

    private final int userId; private final int id;
    private final String title; private final String
    body;

    public Post(
        @JsonProperty("userId") int userId,
        @JsonProperty("id") int id, @JsonProperty("title")
        String title, @JsonProperty("body") String body
    ) {
        this.userId = userId; this.id = id;
        this.title = title; this.body = body;
    }

    public int getUserId() { return userId;
    }
    // ... все getters @Override

    public String toString() { return "Post" +
        "\n userId=" + userId+ "\n id=" + id +
        "\n title=" + title + "\n body=" + body;
    }
}
```

Пояснение:

- Класс Post — это Java-модель, которая описывает структуру одного поста
- Аннотации @JsonProperty указывают, как именно поля из JSON (userId, id, title, body) должны сопоставляться с полями класса
- Таким образом, библиотека Jackson может автоматически преобразовать JSON в объект Post

Класс Post.java

```
[
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere",
  "body": "quia et suscipit..."
},
{
  "userId": 1,
  "id": 2,
  "title": "qui est esse",
  "body": "est rerum tempore vitae\nsequi sint
ni..."
},
{
  "userId": 1,
  "id": 3,
  "title": "ea molestias quasi exercitationem
repellat", "body": "et iusto sed quo
iure\nvoluptatem..."
}
...
{
  "userId": 10,
  "id": 100,
  "title": "at nam consequatur ea labore ea
harum", "body": "cupiditate quo est a modi
nesciunt..."
}
]
```

Пояснение:

- Это пример ответа от удалённого сервера в формате JSON
- Каждый объект массива представляет пост пользователя: содержит userId, уникальный id, заголовок title и текст поста body. Такие данные нужно преобразовать в Java-объекты, чтобы работать с ними удобнее

Преобразование JSON в Java-объекты

```
// Создаём JSON mapper
public static ObjectMapper mapper = new ObjectMapper();
```

Пояснение: `ObjectMapper` — это класс из библиотеки `Jackson`. Он отвечает за преобразование JSON в Java-объекты и обратно. Здесь мы создаём статический объект `mapper`, которым будем пользоваться в коде

```
// Преобразование JSON в список объектов Post
List<Post> posts = mapper.readValue(
    response.getEntity().getContent(),
    new TypeReference<List<Post>>() {}
);
```

Пояснение: метод `readValue` читает JSON-ответ сервера и преобразует его в список объектов типа `Post`. `TypeReference<List<Post>>` нужен, чтобы `ObjectMapper` понял, что мы хотим получить именно коллекцию (`List<Post>`), а не один объект

Преобразование JSON в Java-объекты

```
// Выводим список постов на экран  
posts.forEach(System.out::println);
```

Пояснение: здесь мы проходим по списку постов и выводим каждый объект в консоль. Благодаря методу `toString()` в классе `Post` на экран попадёт удобное строковое представление поста

Преобразование JSON в Java-объекты

```
public class Main {

    public static final String REMOTE_SERVICE_URI = "https://jsonplaceholder.typicode.com/posts";
    public static final ObjectMapper mapper = new ObjectMapper();

    public static void main(String[] args) throws IOException
    {
        CloseableHttpClient httpClient = HttpClientBuilder.create()
            .setUserAgent("My Test Service")
            .setDefaultRequestConfig(RequestConfig.custom()
                .setConnectTimeout(5, TimeUnit.SECONDS)
                .setResponseTimeout(30, TimeUnit.SECONDS)
                .setRedirectsEnabled(false)
                .build())
            .build();

        HttpGet request = new HttpGet(REMOTE_SERVICE_URI); request.setHeader(HttpHeaders.ACCEPT,
            ContentType.APPLICATION_JSON.getMimeType());

        CloseableHttpResponse response = httpClient.execute(request);
        Arrays.stream(response.getAllHeaders()).forEach(System.out::println);

        List<Post> posts = mapper.readValue(response.getEntity().getContent(), new TypeReference<>() {});
        posts.forEach(System.out::println);
    }
}
```


Преобразование JSON в Java-объекты

Пояснения к коду:

1. `REMOTE_SERVICE_URI` — URL удалённого сервиса, к которому отправляется запрос
2. `ObjectMapper` — объект из Jackson для преобразования JSON в Java-объекты
3. Создание клиента (`HttpClientBuilder`) — настраиваем HTTP-клиент (таймауты, заголовки, поведение при редиректах)
4. Создание запроса (`HttpGet`) — формируем GET-запрос на заданный URL, указываем заголовок `Accept: application/json`
5. Выполнение запроса (`execute`) — отправляем запрос на сервер и получаем ответ
6. Вывод заголовков ответа — печатаем все заголовки, которые вернул сервер
7. Парсинг ответа (`mapper.readValue`) — преобразуем тело ответа (JSON) в список объектов `Post`
8. Вывод результата — печатаем объекты `Post` в консоль

Результат запуска программы

```
Headers...
Post
  userId=1 id=1
  title=sunt aut facere repellat provident occaecati
excepturi optio reprehenderit
  body=quia et suscipit suscipit recusandae
consequuntur expedita et
  cum reprehenderit molestiae ut ut quas totam
nostrum rerum
  est autem sunt rem eveniet architecto

Post
  userId=1 id=2
  title=qui est esse
  body=est rerum tempore vitae sequi sint nihil
reprehenderit
  dolor beatae ea dolores neque fugiat blanditiis
voluptate porro vel nihil molestiae ut
reiciendis
  qui aperiam non debitis possimus qui neque nisi
nulla
...
```

Пояснения к результату:

- Headers... — вывод заголовков ответа сервера, которые были напечатаны в консоль
- Post — результат работы метода toString() в классе Post.java. Каждый объект JSON преобразован в Java-объект и выведен на экран
- userId, id, title, body — поля объекта Post, которые соответствуют полям в JSON
- Многоточие (...) означает, что в реальном ответе сервер вернёт больше объектов, но для примера выведены только первые

Практика

Вам пришёл такой JSON-объект от сервера:

```
{  
  "userId": 1,  
  "id": 5,  
  "title": "learn Jackson",  
  "completed": true  
}
```

Какой из следующих Java-классов корректно отобразит его с помощью Jackson и почему?

Практика: варианты

Вариант А

```
public class Post {  
    private int userId;  
    private int id;  
    private String title;  
    private boolean completed;  
    // геттеры/сеттеры  
}
```

Вариант Б

```
public class Post {  
    private int user_id;  
    private int todo_id;  
    private String todo_title;  
    private boolean isCompleted;  
    // геттеры/сеттеры  
}
```

Практика: варианты

Вариант В

```
import com.fasterxml.jackson.annotation.JsonProperty;

public class Post {
    @JsonProperty("userId")
    private int userId;
    @JsonProperty("id")
    private int id;
    @JsonProperty("title")
    private String title;
    @JsonProperty("completed")
    private boolean completed;

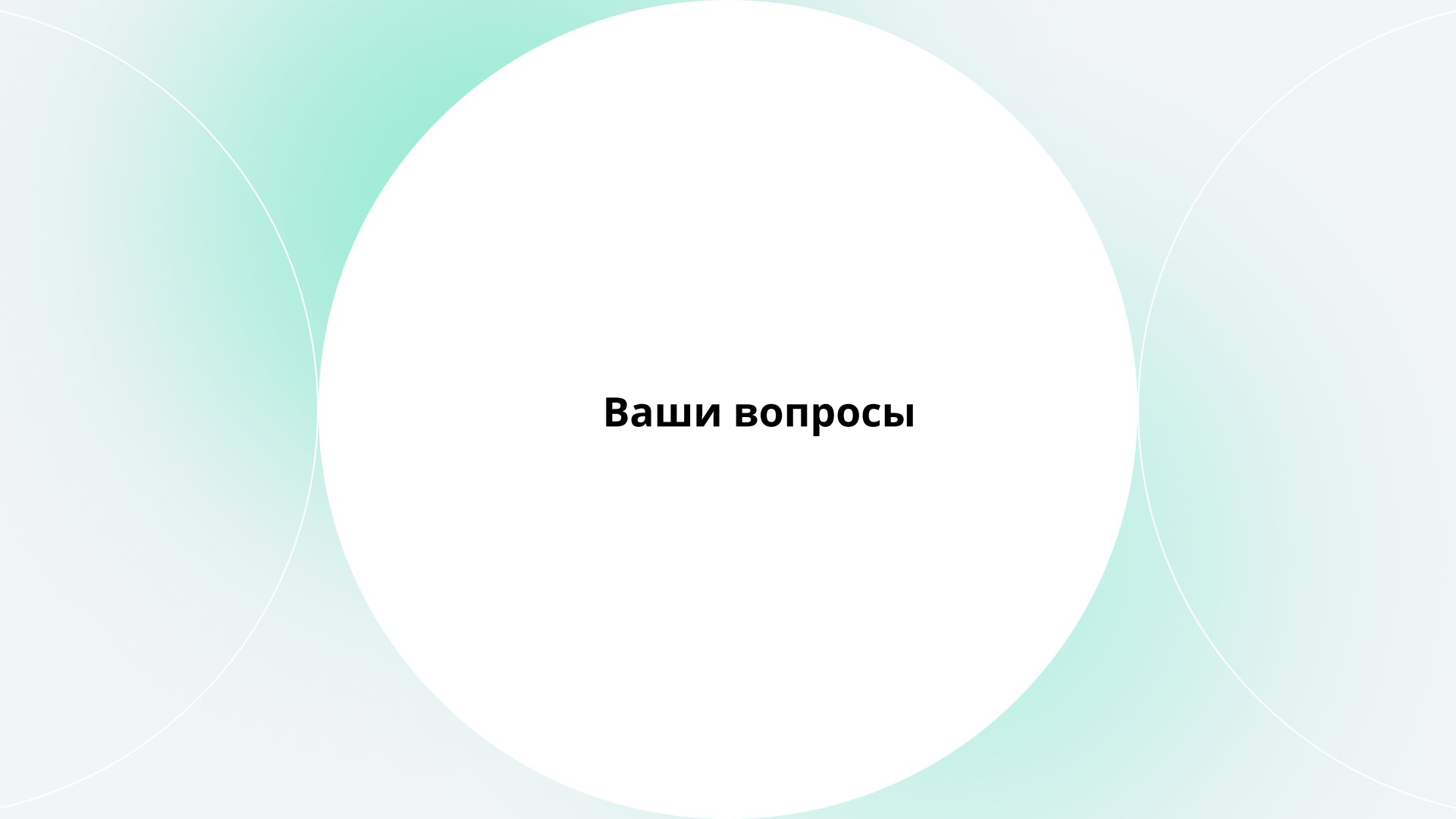
    public Todo(int userId, int id, String title, boolean completed) {
        this.userId = userId;
        this.id = id;
        this.title = title;
        this.completed = completed;
    }
    // только геттеры
}
```

Практика: ответ

Вариант В

Пояснение:

- Jackson по умолчанию маппит поля по точному совпадению имён (case-sensitive)
- Вариант А может работать, если не использовать конструктор и полагаться на сеттеры + публичные поля, но не гарантирует корректную десериализацию при использовании immutable-объектов или строгих настроек Jackson
- Вариант Б — поля не совпадают с JSON → десериализация провалится
- Вариант В — явное указание соответствия через @JsonProperty + конструктор → надёжный и читаемый подход, особенно при final-полях (как в презентации)

The background features a light blue gradient with two large, overlapping teal circles. A large white circle is centered on the page, containing the text.

Ваши вопросы

HTTP и HTTPS

HTTPS

→ **HTTPS** (Hypertext Transport Protocol Secure) — протокол, основанный на HTTP. Обеспечивает безопасный и конфиденциальный обмен данными между сайтом и пользователем

→ **Защита достигается с помощью SSL/TLS и включает три уровня:**

- Шифрование данных — предотвращает перехват информации
- Сохранность данных — любые изменения фиксируются
- Аутентификация — подтверждает подлинность пользователя и сервера

→ Все запросы подписываются сертификатами (ключами), а на стороне получателя данные расшифровываются

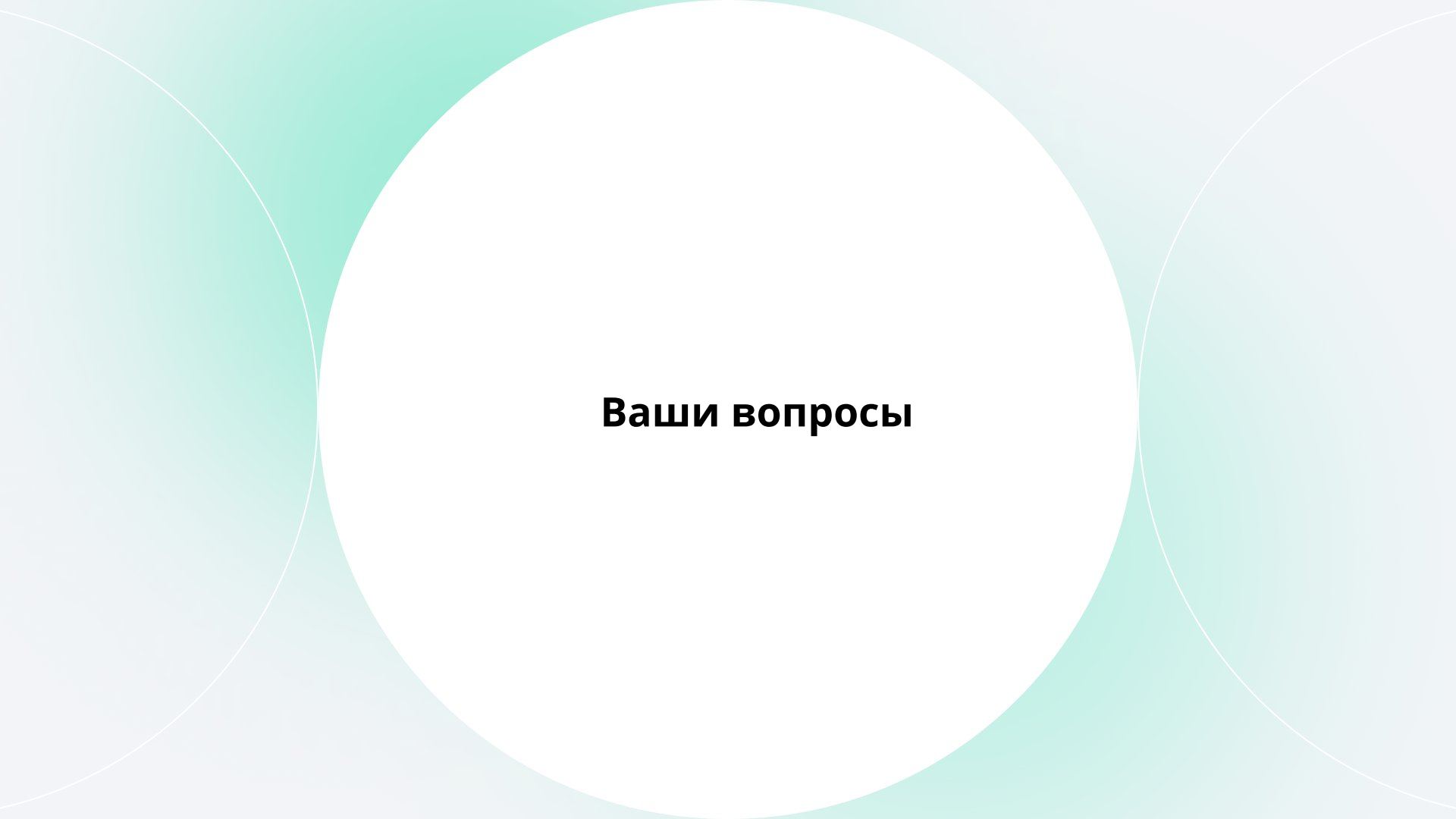
Итоги

Итоги

- Рассмотрели структуру HTTP-запроса и ответа: стартовую строку, заголовки, тело, а также типичные коды состояния и их значение в клиент-серверном взаимодействии
- Узнали, как в Java отправлять HTTP-запросы с помощью современных клиентов, и почему Apache HttpClient 5.x предпочтительнее устаревших решений — благодаря поддержке HTTP/2, Fluent API, асинхронности и улучшенной безопасности
- Выяснили, как преобразовывать JSON-ответ от сервера в Java-объекты с помощью библиотеки Jackson, используя аннотации **@JsonProperty** и класс **ObjectMapper**

Рекомендации

- При работе с внешними API всегда проверяйте заголовки ответа — они содержат важную информацию о типе данных, лимитах, кэшировании и безопасности
- Для новых проектов на Java рекомендуется использовать Apache HttpClient 5.x или встроенный [java.net.http.HttpClient](#) (начиная с Java 11), чтобы избежать технического долга и обеспечить поддержку современных стандартов вроде HTTPS и HTTP/2



Ваши вопросы

Протокол HTTP

Вызовы удаленных серверов

